

EAL/2: Executable engineering arguments and a diagnostic of evidence transfer to language models

Emmett Miller^a

^a*Affiliation and corresponding-author contact to be supplied before journal submission*

Abstract

Engineering conclusions depend on observations, the question asked of a method, its operating conditions and challenges to the inference. EAL/2 makes these dependencies explicit in a versioned argument language with typed method contracts and a bounded support–attack evaluator. We describe its executable profile and examine whether its notation or supplied evidence helps a language model return the correct status of already authored arguments. In a paired diagnostic, one pinned model received meaning-equivalent EAL or JSON representations across five exposed synthetic tasks, eighteen information conditions and two repetitions (180 attempted calls). Strict raw-only correctness was 4/10 for EAL and 5/10 for JSON; a post-hoc readout that ignored extra response fields gave 4/10 and 6/10. For deliberately incorrect proposed answers, relevant raw observations yielded 2/10 and 5/10 correct endpoints, while observations with computed interpreter conclusions yielded 7/10 and 8/10. On irrelevant-record arms, available-information correctness was 3/40 and 26/40 attempts contained detected false support. An earlier relay comparison preserved twelve already correct producer answers with tool evidence versus eight with answer-only transfer, but offered no correction opportunities. These results demonstrate an inspectable implementation and expose failures of model interpretation on the tested cases. They establish neither a notation advantage nor general engineering reasoning ability. We specify the independent-task and measurement controls needed to test those claims.

Keywords: engineering argumentation, domain-specific language, defeasible reasoning, language models, empirical software engineering, evidence transfer

1. Introduction

An engineering decision can turn on a detail that a plausible summary omits. A vibration record may show a value below a limit, yet its calculation can use a different origin from the claim’s formal question. A challenge to an argument can depend on the very claim it challenges, leaving the dispute unresolved until independent grounds are supplied. A status label without its dependencies is difficult to review or revise.

EAL/2 represents claims, observations, assumptions, reasoning methods and objections as named, scoped declarations. Its evaluator checks supplied evidence and method outputs, then computes the status of the resulting support–attack structure. A text model may author or interpret that structure, but its fluent account does not perform the evaluator’s checks. The engineering question is therefore twofold: what can the language and evaluator check, and which parts of their result survive transfer to a model recipient?

This article contributes a precise account of EAL/2’s implemented profile, a worked case that distinguishes numerical success from a matching formal query, and an analysis of two retained model experiments. The first compares hand-offs from an already correct tool-enabled producer. The second introduces deliberately wrong proposed answers and a paired EAL–JSON comparison. The latter tests interpretation of supplied arguments on exposed synthetic tasks, not construction of new engineering arguments. Our central result is bounded: explicit executable dependencies are available, whereas neither a model advantage caused by EAL syntax nor reliable model preservation of qualifications has been shown.

2. Language and executable meaning

2.1. Design boundary

EAL/2 is the repository’s sole supported source language. Its grammar requires the header `language "EAL/2";`; earlier EAL measurements retain their historical versions and cannot be relabelled as EAL/2 results. The ANTLR grammar recognises source, which is lowered and checked against a typed internal representation before evaluation. A `reasoning` declaration selects one versioned method, for example `engineering/rms/1`. The tool’s `mode` instead describes collection variability. Built-in and installed methods use the same contract and result-binding checks. Reusable patterns have

typed parameters, closed lexical scope and a finite expansion into ordinary arguments; expansion retains the identity of shared observations [1].

The language makes an authored inference reviewable, rather than turning prose into a proof. A claim has a statement and an environment. Evidence declares its acquisition tool, kind, scope, age limit and value predicates. Assumptions add validation evidence and optional time bounds. An argument names a conclusion, a reasoning step and the evidence, assumptions or premise claims on which that step depends. An objection names the exact claim, argument, method, assumption or other objection it challenges. Toulmin's separation of grounds, inference and rebuttal informs this vocabulary [2]. SACM defines a metamodel and graphical notation for structured assurance cases [3]. EAL/2 specifies executable method contracts and status evaluation for its own bounded engineering-argument profile; neither specification's existence establishes that one is easier to use. EAL/2 does not implement the complete argument-construction, contrariness or preference machinery of ASPIC+ [4].

2.2. A query that a numerical threshold cannot settle

Listing 1 is a complete, existing EAL/2 programme from the repository. It asks whether recorded root-mean-square (RMS) pressure about zero is at most 5 kPa for a named sample episode. The external method is identified by a versioned contract; the claim's proposition fixes the subject, quantity, unit, scope, time and query. The `binding` clause requires the observation used by the method to answer that formal query.

```
language "EAL/2";
environment lab { require "site" == "bench"; }
tool readings { version "1"; mode deterministic; }
evidence series {
  tool readings;
  kind measurement_series;
  environment lab;
  max_age 3600;
  require "schema" == "EAL/typed-input/1";
}
reasoning rms_method {
  method "engineering/rms/1";
  rationale "Compute RMS deviation from zero over the supplied pressure
    samples.";
}
claim bounded_rms {
```

```

statement "RMS pressure relative to zero is at most 5 kPa for this
  sample.";
environment lab;
proposition {
  subject "pump-A";
  quantity "pressure";
  unit "kPa";
  scope "sample-episode-v1";
  valid_from "2026-09-23T10:00:00Z";
  valid_until "2026-09-23T11:00:00Z";
  query {"origin":0};
  result "rms" <= 5;
}
}
argument sampled_rms {
  conclusion bounded_rms;
  reasoning rms_method;
  evidence series;
  binding series;
}

```

Listing 1: Complete EAL/2 RMS example; reproduced from examples/rms.eal at the repository revision stated in the data availability section.

The diagnostic’s related vibration task supplies samples 0.3 and 0.4 m/s. With origin zero, the computed RMS is $\sqrt{(0.3^2 + 0.4^2)/2} \simeq 0.35355$ m/s, below the 0.36 m/s claim limit. A counterexample retains those samples but records an origin of 0.1 m/s. It yields $\sqrt{((0.3 - 0.1)^2 + (0.4 - 0.1)^2)/2} \simeq 0.25495$ m/s. That smaller result answers a different query. The checked claim is **unsupported** for that supplied observation, even though either scalar passes the numerical threshold [5]. The interpreter’s result is conditional on the authored question, observation identity and method implementation; it does not certify that the physical measurement or model is adequate.

2.3. Support, objections and bounded evaluation

Let $E(a)$, $A(a)$ and $P(a)$ be an argument’s evidence, assumptions and premise claims. Local usability checks declared context and time, evidence acquisition identity and freshness, required predicates, and the method’s typed input and output. The authored argument asserts a conditional relation between those checks and its conclusion. It may have alternative derivations; an observation reused by several arguments retains one identity. The

evaluator rejects cyclic claim-premise dependencies and bounds programme size and graph relationships [6].

Objections form a support–attack graph, including objections against objections. Write $U(n)$ for a node’s local usability, $P(n)$ for its required claims, $A(n)$ for attackers, and $D(c)$ for arguments deriving claim c . Starting with undecided labels, the evaluator monotonically adds justified accepted or rejected labels to its least-information fixed point:

$$\text{Acc}(n) \Leftarrow U(n) \wedge \bigwedge_{c \in P(n)} \text{Acc}(c) \wedge \bigwedge_{a \in A(n)} \text{Rej}(a), \quad (1)$$

$$\text{Rej}(n) \Leftarrow \neg U(n) \vee \bigvee_{c \in P(n)} \text{Rej}(c) \vee \bigvee_{a \in A(n)} \text{Acc}(a), \quad (2)$$

$$\text{Acc}(c) \Leftarrow \bigvee_{n \in D(c)} \text{Acc}(n), \quad \text{Rej}(c) \Leftarrow \bigwedge_{n \in D(c)} \text{Rej}(n). \quad (3)$$

The claim-premise graph is acyclic, while attacks and objection-support relationships may cycle. This is a defined conjunction/disjunction extension whose attack-only restriction agrees with grounded labelling for a finite Dung graph [7, 6]. It does not generate objections from natural-language contradiction or rank competing methods.

For a usable argument, acceptance gives **supported**; a rejected or undecided derivation with no uncontested alternative gives **contested**. With no usable derivation the claim is **unsupported**; a failed environment check gives **out_of_scope**. These labels describe the supplied argument calculus. In particular, **unsupported** does not prove a proposition false. Method execution failure, an unmatched query and an unavailable observation remain distinguishable in the assessment record. Unit, time and scope checks reduce accidental reuse of a computation; engineers still choose the relevant quantities, assumptions and observations.

3. Evaluation design

3.1. Questions and materials

The design claim is that EAL/2 can express and execute the specified engineering dependencies. Executable examples and regression checks exercise that claim within the implemented profile; their pass count does not establish broad representational completeness or human readability. The model questions are different. Does EAL syntax, compared with meaning-equivalent

JSON, improve a pinned recipient’s status answers? Can relevant raw observations correct a wrong proposal without damaging a correct qualified proposal? Does supplying an interpreter conclusion change the answer? A rival explanation of any syntax difference is a prompt-surface effect involving length, tokenisation, familiarity or output compliance, rather than a general reasoning gain [8].

We distinguish the earlier relay pilot from the later notation diagnostic. The relay programme used six previously exposed synthetic tasks, two repetitions, 27 model/transfer conditions and 324 attempted endpoints; 323 completed. In its declared primary contrast, the same GPT-4.1 tool-enabled producer supplied already correct answers on twelve blocks to a GPT-4.1 nano recipient. With tool evidence the recipient retained 12/12; with the producer’s answer alone it retained 8/12. The descriptive task-weighted difference was 33.3 percentage points, with a task-cluster interval of 0.0–66.7 points and an inconclusive prespecified decision. There were no producer errors in those twelve blocks, so this contrast has no correction denominator [9].

The later experiment used five exposed synthetic tasks: a circular defence, an independently grounded defence, a registered RMS calculation, a wrong-origin RMS observation and a sampled negative finding. A source-repair task was excluded because its interface could not be meaningfully matched in this diagnostic. Each task had eighteen conditions and two repetitions, giving 180 scheduled and attempted model calls. The recipient was snapshot `gpt-4.1-nano-2025-04-14`, at temperature zero with a 2,048-token output limit. Each call used a fresh message context, no tools and no repair. EAL and JSON packets had equal parsed meaning, observations and common reference; their natural input lengths differed and the reference used EAL vocabulary. A correct or deliberately incorrect proposed answer was fixed for each task/repetition stratum. The crossed views were answer only, relevant raw observations, irrelevant observations for another assembly, and raw observations plus interpreter conclusions. A fifth view supplied raw observations without a proposed answer [10, 11].

The experimental unit for a notation comparison is a matched task/repetition pair. Two repetitions of the same exposed template are dependent and sometimes contain different wrong proposal labels; they are not ten independent task types. There was no protected unseen-task sample. Assignment, model settings, prompts and references were frozen before generation, with a pre-generation amendment to completion-status scoring and the available-

information reference. After a known-cost output-limit failure at call 30, an operational continuation finished the original schedule without changing prompts or retrying that failure. Its revised stop handling was documented after the event [12]. This departure and task exposure prevent confirmatory population inference.

3.2. Outcomes and estimands

The strict primary score admits a response only if its whole text is a JSON object with exactly **claims** and **basis**, all requested claim identifiers and allowed statuses, and a list-valued basis. Full-information correctness $Y_{ik}^F(n, v, a)$ requires all statuses to match the original complete-observation reference for task i , repetition k , notation n , view v and proposal a . Available-information correctness Y^A compares the same response with the frozen interpreter reference for the records actually supplied. These references coincide for raw-only, relevant-raw and assessed views. For answer-only and irrelevant-record views the interpreter has no relevant observation, so asserting support can agree with the hidden full answer while being unjustified by supplied information.

The finite paired notation contrast is

$$\hat{\Delta}_N = \frac{1}{5} \sum_{i=1}^5 \frac{1}{2} \sum_{k=1}^2 \left[Y_{ik}^F(\text{EAL}, \text{raw}, \emptyset) - Y_{ik}^F(\text{JSON}, \text{raw}, \emptyset) \right]. \quad (4)$$

The corresponding raw-evidence contrast holds the wrong proposal, task and notation fixed and compares answer + raw with answer. It is a response difference under this prompt, not a naturally occurring error-correction rate. Every failed or malformed attempt remains in the correctness denominator. A detected false-support event means that an admitted response marks at least one requested claim **supported** when the selected reference does not. Malformed text and provider errors are excluded from that event count, so its zero would not establish absence of false assertions in unadmitted text.

For an admitted claim map m and a reference o , define $\text{Matches}(m, o) = (m \neq \emptyset) \wedge \bigwedge_{c \in \text{Ids}(o)} (m[c] = o[c])$. Define $\text{FalseSupport}(m, o) = (m \neq \emptyset) \wedge \bigvee_{c \in \text{Ids}(o)} (m[c] = \text{supported} \wedge o[c] \neq \text{supported})$. Algorithm 1 summarises the retained strict scorer and matched contrast procedure using these two exact tests. The implementation additionally verifies frozen schedule and record integrity. The algorithm distinguishes the two reference questions and never repairs malformed model text.

Algorithm 1 Strictly score retained trials against both references and pair notation outcomes

Require: Frozen task/repetition schedule S , attempted records R , full references F , view-specific references V

Ensure: Correctness indicators Y^F, Y^A , detected false support, complete-pair contrasts

```

1: for all scheduled row  $s \in S$  do
2:    $r \leftarrow \text{FINDATTEMPT}(R, s)$ 
3:    $C_s \leftarrow \text{CLAIMIDS}(F_s)$ 
4:    $m \leftarrow \text{ADMITSTRICTCLAIMMAP}(r, C_s)$ 
5:   for all reference  $o \in \{F_s, V_s\}$  do
6:      $Y_s^o \leftarrow \text{MATCHES}(m, o)$ 
7:      $H_s^o \leftarrow \text{FALSESUPPORT}(m, o)$ 
8:   end for
9: end for
10: for all matched task/repetition block  $b$  do
11:   if both raw-only notation attempts are present then
12:      $d_b \leftarrow Y_{b,\text{EAL},\text{raw}}^F - Y_{b,\text{JSON},\text{raw}}^F$ 
13:   end if
14: end for
15: return  $Y^F, Y^A, H, \{d_b\}$ 

```

All 180 calls were attempted once. Eighteen responses failed the exact schema and one hit the output limit; every response retained model identity and usage. The summed reported usage was 1,919,842 input tokens (1,837,824 cached) and 29,816 output tokens. Configured-rate cost was USD 0.0660738, an estimate rather than an invoice. The archive stores raw records, hashes, scoring tables and the continuation record [11].

4. Results

Table 1 retains the attempt denominator and separates the full and supplied-information references. The strict raw-only notation comparison yielded EAL 4/10 and JSON 5/10, or $\hat{\Delta}_N = -0.10$. Four matched pairs were correct in both formats, one only in JSON and five in neither. No task showed an EAL-only success; the single discordant pair arose in the sampled-negative-finding task. This describes these ten paired attempts and does not estimate a population JSON advantage.

Table 1: Strict complete correctness, with ten attempted calls per notation and view. Correct and incorrect describe the injected proposal against the full-information answer.

View and proposal	EAL full	JSON full	EAL available	JSON available
Raw only; no proposal	4/10	5/10	4/10	5/10
Correct; answer only	10/10	9/10	2/10	2/10
Correct; relevant raw	2/10	7/10	2/10	7/10
Incorrect; answer only	0/10	2/10	4/10	4/10
Incorrect; relevant raw	2/10	5/10	2/10	5/10
Incorrect; assessed	7/10	8/10	7/10	8/10
Correct; irrelevant	6/10	6/10	0/10	0/10
Incorrect; irrelevant	2/10	3/10	3/10	0/10

With a deliberately wrong proposal, relevant raw records produced two EAL and five JSON complete corrections, compared with zero and two for answer-only. Supplying interpreter conclusions along with the raw records raised complete correctness to seven and eight. These differences involve supplied information as well as changes in prompt length, and cannot isolate independent calculation by the recipient. In the correct-proposal stratum, EAL answer-plus-raw scored only 2/10 versus 10/10 for answer only; JSON scored 7/10 versus 9/10. Under the supplied-information reference, answer-only correctness was 2/10 for a full-information-correct proposal in either notation, because the proposal itself supplied no observation. Describing that reference change as damage to correct empirical support would invert the evidence requirement.

The irrelevant-record control exposes a separate failure. Across its four representation/proposal arms, only 3/40 attempts were correct for information actually available, and 26/40 contained detected false support in admitted responses. None of the 28 proposals wrong under that reference was corrected. Values from another assembly could appear numerically plausible, but they were not observations for the requested scope. In raw-only trials both representations failed the circular-defence and wrong-origin tasks in both repetitions; each succeeded on the independently grounded defence and matching RMS task twice [11].

The prespecified strict output contract counts the eighteen malformed responses as failures. A later, explicitly exploratory readout accepts a strictly parsed raw JSON claim map with extra top-level fields and ignores the `basis` shape. It changes raw-only results to EAL 4/10 and JSON 6/10. It

also increases detected false-support events across all conditions from 69/180 to 77/180. Fence-only normalisation changed no primary count. Relaxing output shape therefore cannot reveal an EAL advantage in this sample; it changes the measured endpoint and was chosen after schema failures had been seen [11].

5. Interpretation and limitations

The implemented checks have a clear use: an engineer can ask which observation, query, scope, assumption or challenge caused a status and can reassess after a recorded change. Typed method contracts prevent a numerically passing result for the wrong formal query from silently supporting the claim. That is a software behaviour claim supported by the specified semantics and repository checks. Whether the authored relation from a measurement to a physical conclusion is sound still requires domain evidence. Reusing the same observation through two arguments does not create two independent measurements; the calculus does not assign numerical credibility to agreement among arguments.

What can the model trials establish about the notation itself? In this diagnostic they establish only a finite paired outcome for one model on five exposed tasks. Meaning-equivalent EAL and JSON differ in tokenisation and length, and the common reference uses EAL terms. Even a larger observed difference in this design would combine those surface mechanisms. Prompt-format sensitivity elsewhere supplies a reason to compare plausible formats; it does not predict EAL’s direction of effect [8]. The present strict and exploratory comparisons give no positive EAL notation result. A fully structured JSON interchange remains a credible design alternative for the same typed representation.

Computed interpreter conclusions are different from raw measurements. Their larger correction counts show that the tested recipient can often use a supplied status; they do not show that it independently applied the method or checked the scope. The earlier relay benefit likewise concerns preservation of a correct producer’s qualified answer. Together the experiments separate preservation, recovery from injected wrong labels and independent verification. This separation is more useful than a pooled reasoning-accuracy number because each property requires a different initial-answer and evidence condition.

Several limits affect inference. The five diagnostic task templates and six relay templates were public development material, and related cases share mechanisms. Repetitions are dependent; task-cluster intervals from six clusters have poor coverage in the repository’s sensitivity simulation [9]. A failed schema or output limit counts against an end-to-end answer but does not reveal the model’s latent status judgement. The short public justifications are archived, yet no independently validated quality score of those explanations is available. A correct interpreter answer is conditional on its source, method contracts and reference fixtures, not a measurement of real plant behaviour. Finally, the after-call continuation and post-hoc format readout must remain visible rather than being treated as prespecified confirmatory analyses. Repeated adaptive use of an exposed task set can reward local tuning and weaken a generalisation claim [13].

An independent test should sample new mechanisms across electronics and control, mechanical and thermal systems, distributed software, and reliability or manufacturing. Task authors should check the formal and engineering answers independently, including underdetermined cases. Four matched conditions—EAL and JSON, each with or without the same interpreter operations—would separate surface rendering from executable assistance. Freeze the task frame, model snapshot, prompts, budgets, primary status and qualification scores, and an information target before revealing outcomes. Match tool access and method feedback, retain failed attempts, audit false support by scope, and assess human comprehension separately. Such a study could support a model- and task-bounded causal notation claim if assignment, measurement and coverage checks hold. It would still require external engineering cases before claims about field use.

6. Conclusion

EAL/2 supplies a finite, executable account of named argument dependencies, typed method results and targeted objections. Its retained model trials show that already computed conclusions often transfer better than raw records on the exposed cases, while raw evidence and irrelevant-record handling remain unreliable for this recipient. Strict EAL raw-only correctness was 4/10 against 5/10 for meaning-equivalent JSON; the relaxed exploratory readout was 4/10 against 6/10. The defensible engineering use is to retain and inspect the interpreter’s dependencies and qualifications when a model communicates a conclusion. An independent task sample and balanced tool

comparison are required to establish a notation or general reasoning advantage.

Data and code availability

The source, protocols, raw trial archive, hashes and analysis scripts are retained at revision 2b0815f111f831bf237f600cf9e65f28eca0403e of <https://github.com/emmett08/earl>. This repository was private when this manuscript was prepared; access for external reviewers and a public archival identifier remain to be arranged by the author. The archived analysis can be rerun without model calls using `scripts/analyse_notation_transfer.py` and `scripts/analyse_notation_format_sensitivity.py`. The relay record is under `benchmarks/results/2026-09-23-relays-recovery/`; the notation record is under `benchmarks/results/2026-09-23-notation-transfer-live/`.

Funding

The author must supply a factual funding statement before submission.

Declaration of competing interest

The author must supply a factual competing-interest declaration before submission.

Author contributions

The author must verify the work and supply a CRediT contribution statement before submission. An AI system is not an author.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

OpenAI ChatGPT (Codex) assisted with manuscript drafting, source checking, data-summary checking and LaTeX preparation. The human author must review and edit the content, verify every claim and reference, and take responsibility for the submitted article.

References

- [1] E. project, Eal/2 design decisions and language reference, Repository documentation, revision 2b0815f, private repository at manuscript preparation (2026).
URL <https://github.com/emmett08/earl/blob/2b0815f111f831bf237f600cf9e65f28eca0403e/docs/eal2-design.md>
- [2] S. E. Toulmin, The Uses of Argument, updated edition Edition, Cambridge University Press, 2003. doi:10.1017/CB09780511840005.
- [3] Object Management Group, Structured assurance case metamodel, version 2.2, OMG specification, formal/21-11-01 (2021).
URL <https://www.omg.org/spec/SACM/2.2/About-SACM>
- [4] S. Modgil, H. Prakken, The ASPIC+ framework for structured argumentation: a tutorial, *Argument & Computation* 5 (1) (2014) 31–62. doi:10.1080/19462166.2013.869766.
- [5] E. project, Engineering task suite, version engineering-v2.1-eal2, Repository fixture, revision 2b0815f, private repository at manuscript preparation (2026).
URL <https://github.com/emmett08/earl/blob/2b0815f111f831bf237f600cf9e65f28eca0403e/benchmarks/engineering-v2/suite.json>
- [6] E. project, Engineering argument and reasoning model, Repository documentation, revision 2b0815f, private repository at manuscript preparation (2026).
URL <https://github.com/emmett08/earl/blob/2b0815f111f831bf237f600cf9e65f28eca0403e/docs/argument-model.md>
- [7] P. M. Dung, On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games, *Artificial Intelligence* 77 (2) (1995) 321–357. doi:10.1016/0004-3702(94)00041-X.
- [8] M. Sclar, Y. Choi, Y. Tsvetkov, A. Suhr, Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting, in: International Conference

on Learning Representations, 2024.

URL <https://openreview.net/forum?id=RIu5lyNXjT>

- [9] E. project, Eal/2 model-sequence results, Repository report, revision 2b0815f, private repository at manuscript preparation (2026).
URL <https://github.com/emmett08/earl/blob/2b0815f111f831bf237f600cf9e65f28eca0403e/docs/eal2-relay-results.md>
- [10] E. project, Inv-eal-reasoning-001 investigation protocol, version 1.0.1, Repository protocol, revision 2b0815f, private repository at manuscript preparation (2026).
URL <https://github.com/emmett08/earl/blob/2b0815f111f831bf237f600cf9e65f28eca0403e/benchmarks/protocols/INV-EAL-REASONING-001.json>
- [11] E. project, Eal/json notation and evidence-transfer diagnostic, Repository report and raw records, revision 2b0815f, private repository at manuscript preparation (2026).
URL <https://github.com/emmett08/earl/blob/2b0815f111f831bf237f600cf9e65f28eca0403e/docs/notation-transfer-live-results.md>
- [12] E. project, Notation-transfer execution record, Repository record, revision 2b0815f, private repository at manuscript preparation (2026).
URL <https://github.com/emmett08/earl/blob/2b0815f111f831bf237f600cf9e65f28eca0403e/benchmarks/results/2026-09-23-notation-transfer-live/execution-notes.md>
- [13] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, A. Roth, Generalization in adaptive data analysis and holdout reuse, in: Advances in Neural Information Processing Systems, Vol. 28, 2015.
URL <https://proceedings.neurips.cc/paper/2015/hash/bad5f33780c42f2588878a9d07405083-Abstract.html>